

**INSTITUTO FEDERAL DA BAHIA
CAMPUS FEIRA DE SANTANA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO**

GUSTAVO DA SILVA FERREIRA

**PERCEPÇÕES E ADOÇÃO DE TYPESCRIPT E JAVASCRIPT NO
DESENVOLVIMENTO FRONT-END: UM ESTUDO ENTRE INICIANTE E
PROFISSIONAIS EXPERIENTES**

**FEIRA DE SANTANA-BA
2025**

GUSTAVO DA SILVA FERREIRA

**PERCEPÇÕES E ADOÇÃO DE TYPESCRIPT E JAVASCRIPT NO
DESENVOLVIMENTO FRONT-END: UM ESTUDO ENTRE INICIANTES E
PROFISSIONAIS EXPERIENTES**

Trabalho de Conclusão de Curso submetido ao Curso de Bacharelado em Sistemas de Informação, do campus de Feira de Santana do Instituto Federal da Bahia (IFBA), como um dos requisitos para a obtenção do título de Bacharel em Sistemas de Informação

Orientador: Prof Dr.Clerber Jorge Lira

**FEIRA DE SANTANA-BA
2025**

Percepções e Adoção de TypeScript e JavaScript no Desenvolvimento Front-End: Um Estudo entre Iniciantes e Profissionais Experientes

Resumo

O desenvolvimento de aplicações front-end modernas demanda ecossistemas que equilibrem agilidade de entrega, facilidade de manutenção e confiabilidade de código. Nesse cenário, o debate entre o uso de JavaScript (JS), com sua tipagem dinâmica tradicional, e de TypeScript (TS), com seu sistema de tipagem estática opcional, tornou-se central na engenharia de software. Este Trabalho de Conclusão de Curso investiga as percepções e os fatores determinantes na adoção de JavaScript e TypeScript, analisando comparativamente as diferenças entre desenvolvedores iniciantes e profissionais experientes. A pesquisa adotou uma abordagem mista (quantitativa e qualitativa) por meio de um estudo de levantamento (*survey*) aplicado a 23 desenvolvedores, contemplando desde iniciantes até especialistas com vivência em projetos corporativos de grande escala. Os dados foram analisados através de estatística descritiva e categorização temática de respostas abertas. Os resultados revelam uma marcante divergência cognitiva condicionada pela maturidade profissional: enquanto iniciantes priorizam a flexibilidade e a prototipagem rápida do JavaScript (média de 4,29 na escala Likert), profissionais experientes manifestam ganhos expressivos de produtividade com o TypeScript (média de 4,22 contra 2,93 de iniciantes) e esmagadora preferência por sua adoção em novos projetos corporativos (4,56). Ambos os grupos convergem no reconhecimento de que a ausência de tipagem no JavaScript é propensa à geração de falhas em tempo de execução (média de 4,17) e que o TypeScript aprimora substancialmente a detecção precoce de erros (4,17 geral e 4,78 entre seniores), a manutenibilidade a longo prazo (4,44) e a padronização em equipes multidisciplinares (4,30). Constata-se que a decisão de adoção transcende aspectos puramente sintáticos, configurando-se como uma escolha arquitetural estratégica influenciada pela escala do projeto e pela experiência da equipe de desenvolvimento.

Palavras-chave: JavaScript. TypeScript. Desenvolvimento Front-End. Tipagem Estática. Manutenibilidade de Software. Engenharia de Software.

Perceptions and Adoption of TypeScript and JavaScript in Front-End Development: A Study
Among Beginners and Experienced Professionals

Abstract

The development of modern front-end applications demands ecosystems that balance delivery agility, maintainability, and code reliability. In this scenario, the debate between JavaScript (JS), characterized by dynamic typing, and TypeScript (TS), featuring an optional static typing system, has become central in software engineering. This Final Graduation Project investigates the perceptions and key factors influencing the adoption of JavaScript and TypeScript, comparatively analyzing differences between novice and experienced developers. The research employed a mixed-methods survey approach applied to 23 front-end developers, ranging from beginners to specialists with extensive experience in large-scale corporate projects. Data were processed using descriptive statistics and thematic categorization of open-ended qualitative responses. The findings demonstrate a notable perceptual divergence conditioned by professional seniority: while beginners prioritize JavaScript's flexibility and rapid prototyping capabilities (mean of 4.29 on a 5-point Likert scale), experienced professionals report substantial productivity gains with TypeScript (mean of 4.22 vs. 2.93 for beginners) and an overwhelming preference for adopting TypeScript in new projects (4.56). Both cohorts agree that the absence of static typing in JavaScript is prone to runtime errors (mean of 4.17) and that TypeScript substantially improves early bug detection (4.17 overall and 4.78 among seniors), long-term maintainability (4.44), and team code standardization (4.30). The study concludes that technology adoption transcends mere syntactic preference, representing a strategic architectural choice determined by project scale and team maturity.

Keywords: JavaScript. TypeScript. Front-End Development. Static Typing. Software Maintainability. Software Engineering.

Lista de figuras

Figura 1 – Perfil dos respondentes por tempo de experiência e participação em grandes projetos	9
Figura 2 – Comparativo de proficiência técnica autoavaliada em JavaScript e TypeScript	10
Figura 3 – Comparativo de médias no Bloco A entre Iniciantes e Experientes	11
Figura 4 – Comparativo de médias no Bloco B entre Iniciantes e Experientes	13
Figura 5 – Comparativo de médias no Bloco C entre Iniciantes e Experientes	15
Figura 6 – Comparativo de médias no Bloco D entre Iniciantes e Experientes	17
Figura 7 – Divergência perceptual consolidada entre Experientes e Iniciantes (Δ de Médias)	18

Lista de tabelas

Tabela 1 – Estatísticas descritivas do Bloco A: Produtividade e Desenvolvimento . . .	11
Tabela 2 – Estatísticas descritivas do Bloco B: Qualidade e Manutenção do Código . .	13
Tabela 3 – Estatísticas descritivas do Bloco C: Compatibilidade e Ferramentas	15
Tabela 4 – Estatísticas descritivas do Bloco D: Preferências e Perspectivas Pessoais . .	16
Tabela 5 – Cronograma de atividades do TCC	25

Lista de quadros

Quadro 1 – Confronto entre a literatura acadêmica e as evidências empíricas da pesquisa 21

Sumário

1 – Introdução	1
1.1 Justificativa	2
1.2 Objetivos	2
1.2.1 Objetivo Geral	2
1.2.2 Objetivos Específicos	2
2 – Fundamentação Teórica	3
3 – Metodologia	6
4 – Análise e Discussão dos Resultados	8
4.1 Caracterização Demográfica dos Participantes	8
4.2 Produtividade e Desenvolvimento Ágil (Bloco A)	10
4.3 Qualidade do Código, Manutenibilidade e Prevenção de Falhas (Bloco B)	12
4.4 Compatibilidade, Ferramentas e Desafios de Migração (Bloco C)	14
4.5 Preferências Pessoais, Confiança e Perspectivas Futuras (Bloco D)	16
4.6 Análise Qualitativa dos Relatos Práticos da Indústria	18
4.6.1 Eixo 1: Blindagem de Contratos de Dados e Integração de APIs	18
4.6.2 Eixo 2: Refatoração em Larga Escala e Propagação de Modificações	19
4.6.3 Eixo 3: Complexidade Inicial e Fricção com o Ecossistema Externo	19
4.6.4 Eixo 4: A Fronteira entre Compilação e Execução (Runtime vs Compile-time)	20
4.7 Confronto Teórico-Empírico e Síntese da Discussão	20
5 – Considerações Finais	22
5.1 Síntese dos Resultados e Atendimento aos Objetivos	22
5.2 Contribuições e Recomendações Práticas	23
5.2.1 Diretrizes para a Indústria e Gestores de Tecnologia	23
5.2.2 Diretrizes para o Ensino Acadêmico de Computação	23
5.3 Limitações da Pesquisa	23
5.4 Sugestões para Trabalhos Futuros	24
Referências	26

1 Introdução

O desenvolvimento de aplicações para a Web representa, nas últimas décadas, um dos setores mais dinâmicos e influentes da tecnologia da informação. A crescente demanda por experiências digitais ricas, interativas e de alto desempenho impulsionou uma rápida evolução das tecnologias de front-end, a camada da aplicação com a qual o usuário final interage diretamente. Nesse ecossistema, a linguagem JavaScript (JS) se consolidou como pilar tecnológico, sendo, por mais de uma década, a mais utilizada por desenvolvedores em todo o mundo ([Stack Overflow](#), 2023).

Originalmente concebido para adicionar interatividade a páginas estáticas, o JavaScript evoluiu, por meio da padronização do *ECMAScript*, para uma linguagem versátil e poderosa. Contudo, a mesma flexibilidade que impulsionou sua popularidade — notadamente a tipagem dinâmica — pode ampliar a distância entre o código produzido e os problemas observados em execução. Estudos sobre defeitos em JavaScript indicam que uma parcela desses problemas poderia ser detectada por verificações estáticas, embora isso não esgote as causas de falhas em aplicações web ([Makimov](#), 2022; [Richards](#); [Nardelli](#); [Vitek](#), 2017).

Em resposta a essa lacuna, a Microsoft lançou, em 2012, o TypeScript (TS), um superconjunto de JavaScript que adiciona tipagem estática gradual e é posteriormente transformado em JavaScript. Essa arquitetura favorece a migração incremental e amplia o suporte do editor a navegação, autocompletar e refatoração, mas não constitui uma garantia de correção em tempo de execução. A literatura mais recente sugere, inclusive, que o efeito da linguagem depende da configuração do compilador e da qualidade dos contratos mantidos nas bibliotecas externas ([George](#), 2020; [Zandonotto et al.](#), 2026; [Feldthaus](#); [Møller](#), 2014).

Assim, o TypeScript não deve ser compreendido como uma solução que elimina bugs, mas como uma tecnologia que altera o perfil dos riscos. A análise de defeitos da própria linguagem identificou problemas envolvendo tipos, semântica, escopos, recursos e comportamento do compilador ([Wang](#); [Fang](#); [Wang](#), 2025). Em projetos reais, falhas de configuração, uso de APIs, dependências e assincronicidade também podem superar os erros locais de lógica, deslocando parte da fragilidade para as fronteiras entre código, ferramentas e ambiente de execução ([Tang](#); [Alimadadi](#); [Sumner](#), 2026).

A escolha entre JavaScript e TypeScript envolve um balanço complexo de vantagens e desvantagens, percebidas de maneiras distintas por desenvolvedores com diferentes níveis de experiência. Enquanto profissionais experientes tendem a valorizar a robustez e a estrutura que o TypeScript oferece, iniciantes podem ser atraídos pela simplicidade e pela curva de aprendizado mais suave do JavaScript ([Shrestha et al.](#), 2022).

1.1 Justificativa

Este estudo se justifica pela sua dupla relevância, prática e acadêmica. No contexto profissional, uma compreensão aprofundada dos fatores que influenciam a adoção de cada linguagem pode subsidiar decisões estratégicas de equipes e gestores de tecnologia. A escolha da ferramenta adequada impacta diretamente a produtividade, a manutenibilidade de projetos e a capacidade de escalar sistemas de forma sustentável.

Do ponto de vista acadêmico, embora a popularidade do TypeScript e os possíveis ganhos da tipagem estática sejam bem documentados, ainda é necessário relacionar esses ganhos às condições concretas de adoção. A configuração do compilador, a correspondência entre interfaces e bibliotecas JavaScript e a complexidade do toolchain influenciam o resultado obtido. Há também uma lacuna sobre como a percepção desses benefícios e desafios é modulada pelo nível de experiência do desenvolvedor, especialmente no cenário tecnológico brasileiro. Esta pesquisa busca contribuir para esse debate, examinando a escolha entre JavaScript e TypeScript a partir da literatura e das percepções de desenvolvedores.

1.2 Objetivos

1.2.1 Objetivo Geral

Investigar as percepções e os fatores que influenciam a adoção de JavaScript e TypeScript no desenvolvimento front-end, analisando as diferenças entre desenvolvedores iniciantes e profissionais experientes.

1.2.2 Objetivos Específicos

1. Analisar comparativamente as características técnicas do JavaScript e do TypeScript, com foco nos sistemas de tipagem e seus impactos no desenvolvimento.
2. Identificar, com base na literatura e em dados, os benefícios e desafios associados à adoção do TypeScript em projetos de software.
3. Avaliar de que forma o nível de experiência dos desenvolvedores (iniciantes versus experientes) influencia a percepção e a preferência por uma ou outra tecnologia.

2 Fundamentação Teórica

A presente análise teórica explora os conceitos que fundamentam a coexistência entre JavaScript e TypeScript no desenvolvimento de aplicações web. O percurso argumentativo parte da tipagem dinâmica e da assincronicidade do JavaScript, apresenta o TypeScript como uma forma de tipagem gradual e discute os limites dessa proteção. Em seguida, examina contratos de bibliotecas, configuração do compilador, ferramentas de migração e a mudança do perfil de falhas em ecossistemas modernos, posicionando a experiência do desenvolvedor como um fator relevante, mas não isolado, na escolha tecnológica.

O desenvolvimento front-end foi profundamente moldado pelo papel central do *JavaScript*, linguagem criada em 1995 por Brendan Eich. Embora concebido inicialmente como um recurso de script para adicionar interatividade a páginas estáticas, sua evolução, marcada pela padronização como *ECMAScript*, o consolidou como a principal tecnologia para aplicações web, garantindo consistência entre diferentes ambientes de execução e tornando-se o padrão universal na camada de interação com o usuário (Mororó, 2024). Uma de suas características mais marcantes é a tipagem dinâmica, na qual os tipos de dados estão associados aos valores, e não às variáveis. Essa flexibilidade permite uma escrita de código ágil, tornando a linguagem atrativa para prototipagem e projetos menores, onde uma mesma variável pode mudar de tipo dinamicamente sem que o desenvolvedor precise de declarações prévias.

```
let resultado = 100; // 'resultado' é inferido como number
resultado = 'cem';   // Válido em JS. Agora 'resultado' é uma string.
```

Contudo, essa mesma liberdade introduz uma classe de desafios em sistemas complexos. Como aponta (Makimov, 2022), erros de tipo, como a passagem de dados em formato inesperado para uma função, podem só se manifestar em tempo de execução. Tal característica eleva o custo de manutenção, pois a depuração de erros em grandes bases de código torna-se mais árdua. A essa complexidade soma-se o modelo de concorrência do JavaScript, baseado em um *event loop*. Embora eficaz para impedir que operações demoradas bloqueiem a interface, a gestão do fluxo assíncrono permanece uma fonte de condições de corrida, falhas de tratamento e comportamentos difíceis de reproduzir. A introdução de *Promises* e, posteriormente, da sintaxe *async/await*, melhorou a legibilidade, mas não substituiu a necessidade de testes e validações em execução (Mororó, 2024; Tang; Alimadadi; Sumner, 2026).

Foi nesse cenário de um ecossistema poderoso, porém propenso a desafios de escalabilidade, que a Microsoft lançou o *TypeScript* em 2012. Definido como um superconjunto de JavaScript, sua principal inovação foi a introdução de um sistema de tipagem estática opcional. Essa abordagem significa que todo código JavaScript é, por definição, um código TypeScript

válido, o que viabiliza uma migração gradual e estratégica. A proposta central do TypeScript é deslocar a verificação de tipos do tempo de execução para o tempo de compilação, proporcionando maior segurança e melhorando a legibilidade, visto que o código se torna, em parte, autodescritivo (George, 2020; Makimov, 2022). Um erro de atribuição de tipo, por exemplo, é capturado antes mesmo que o programa seja executado.

```
let meuNome: string = "Gustavo";  
// O compilador do TypeScript apontará um erro nesta linha:  
// Erro: O tipo 'number' não pode ser atribuído ao tipo 'string'.  
meuNome = 30;
```

A promessa de maior robustez do TypeScript deve, portanto, ser interpretada de forma condicional. Um estudo de (Richards; Nardelli; Vitek, 2017) estimou que a tipagem estática poderia prevenir cerca de 15% dos bugs públicos em projetos JavaScript. Por outro lado, a análise de (Bogner; Wagner, 2022) indicou que aplicações TypeScript podem apresentar melhor qualidade de código sem necessariamente possuir menos bugs. Esse resultado é compatível com a ideia de que a tipagem reduz determinadas falhas locais, mas não substitui a análise da lógica, dos requisitos e das integrações.

Essa distinção é reforçada por estudos sobre o próprio TypeScript. Wang, Fang e Wang (Wang; Fang; Wang, 2025) identificaram uma variedade de defeitos ligados a nomes, bindings, escopos, tipos, semântica, recursos da linguagem e comportamento do compilador. Em uma escala mais ampla, Tang, Alimadadi e Sumner (Tang; Alimadadi; Sumner, 2026) analisaram 633 relatos de bugs em 16 projetos e observaram a predominância de problemas de configuração e ferramentas, uso incorreto de APIs e tratamento assíncrono. Os resultados não autorizam concluir que o TypeScript seja menos confiável; indicam que a adoção desloca parte da atenção necessária para as camadas de integração e orquestração.

Para além da tipagem, o TypeScript oferece interfaces, tipos genéricos e mecanismos de composição que podem funcionar como documentação e apoio à refatoração. Entretanto, uma declaração de tipo só é útil quando representa o comportamento real do componente. Feldthaus e Møller (Feldthaus; Møller, 2014) verificaram a correção de interfaces TypeScript para bibliotecas JavaScript e demonstraram a importância de testar o contrato contra a implementação. De modo semelhante, Hoefflich, Findler e Serrano (Hoefflich; Findler; Serrano, 2022) desenvolveram o Scotty para encontrar incompatibilidades no repositório DefinitelyTyped, mostrando que declarações incorretas podem produzir uma sensação indevida de segurança. A fronteira entre o tipo declarado e o comportamento efetivo da biblioteca é, portanto, um elemento central da confiabilidade.

Apesar dos benefícios, a adoção do TypeScript impõe uma curva de aprendizado e custos de configuração. Desenvolvedores precisam compreender novos conceitos, integrar compi-

lador, bundler, testes e linters e decidir quanto rigor aplicar ao projeto. Em estudo com mais de 4.000 repositórios, Zandonotto et al. ([Zandonotto et al., 2026](#)) encontraram associação entre opções como `noImplicitAny` e `noUnusedParameters` e menores índices de problemas estáticos, mas também observaram correlações contraintuitivas para algumas opções. Como o estudo é correlacional, seus resultados não constituem garantia causal; ainda assim, evidenciam que a simples presença de arquivos `.ts` não define o nível de proteção obtido.

A migração pode ser apoiada por ferramentas de inferência e geração automática. O dataset `ManyTypes4TypeScript` reúne projetos e anotações de tipos em escala para pesquisas de inferência ([Jesse; Devanbu, 2022](#)), enquanto Jesse, Devanbu e Sawant ([Jesse; Devanbu; Sawant, 2023](#)) investigaram a previsão de tipos definidos por usuários. Cristiani e Thiemann ([Cristiani; Thiemann, 2021](#)) estudaram a geração de arquivos de declaração a partir de JavaScript. Essas técnicas podem reduzir o custo inicial, mas não eliminam a revisão humana: uma inferência ou declaração incorreta pode propagar um contrato falso pelo sistema.

Como aponta ([Shrestha et al., 2022](#)), o conhecimento prévio de JavaScript também pode interferir na adaptação ao novo paradigma. Ainda assim, a integração com frameworks e ferramentas modernas e os ganhos percebidos em projetos de grande porte ajudam a explicar a relevância do TypeScript ([Stack Overflow, 2023](#)). A literatura, portanto, não sustenta uma oposição simples entre uma linguagem melhor e outra pior. O JavaScript permanece vantajoso para aprendizagem, prototipagem e situações em que a agilidade inicial é prioritária; o TypeScript tende a oferecer maior apoio à manutenção e à coordenação em bases extensas, desde que acompanhado por contratos confiáveis, configuração coerente, testes e validações de runtime. Esse equilíbrio orienta a análise empírica deste trabalho.

3 Metodologia

Para atingir os objetivos propostos neste trabalho, notadamente a investigação das percepções e dos fatores que influenciam a adoção de JavaScript e TypeScript entre diferentes perfis de desenvolvedores, optou-se por uma abordagem de pesquisa quantitativa, caracterizada como um estudo de levantamento (survey). Este método é particularmente eficaz para coletar dados padronizados de uma amostra representativa de uma população, permitindo a análise de tendências, padrões e correlações, conforme discutido em metodologias de pesquisa na ciência da informação e computação (Zimmermann, 2010; Pedrycz; Gomide, 1998). A escolha por um levantamento se justifica pela necessidade de capturar uma variedade de perspectivas de forma estruturada, possibilitando uma comparação direta entre os grupos de interesse.

O instrumento de coleta de dados consistiu em dois questionários distintos, elaborados e distribuídos por meio da plataforma Google Forms. A decisão de criar dois formulários separados foi estratégica, visando adequar a linguagem e o foco das perguntas ao nível de conhecimento e ao contexto de cada um dos públicos-alvo. Ambos os questionários foram estruturados com perguntas fechadas, utilizando escalas de Likert (por exemplo, de "discordo totalmente" a "concordo totalmente") e perguntas de múltipla escolha, para facilitar a análise estatística posterior e garantir a objetividade na coleta das percepções.

O primeiro grupo-alvo foi composto por **desenvolvedores profissionais experientes**, atuantes no mercado de desenvolvimento de software. A amostra para este grupo foi selecionada por conveniência, utilizando redes de contatos profissionais, como o LinkedIn, e comunidades online de desenvolvedores. O critério para ser considerado "experiente" foi definido como possuir dois ou mais anos de experiência profissional comprovada no desenvolvimento front-end. Este grupo é fundamental para compreender as percepções sobre as linguagens em um contexto de projetos comerciais, de longa duração e com equipes colaborativas, onde questões como manutenibilidade e escalabilidade, frequentemente citadas na literatura (Makimov, 2022; George, 2020), são de suma importância.

O segundo grupo-alvo foi formado por **desenvolvedores iniciantes**, majoritariamente estudantes do curso de Bacharelado em Sistemas de Informação do Instituto Federal da Bahia (IFBA) - Campus Feira de Santana, e de outras instituições de ensino superior. A amostra também foi selecionada por conveniência, através de convites diretos em grupos de estudo e canais de comunicação acadêmicos. Este grupo representa a nova geração de desenvolvedores, cujas percepções são moldadas pelas primeiras experiências de aprendizado e pelo contato inicial com as tecnologias. A análise deste grupo é crucial para investigar fatores como a curva de aprendizado e a simplicidade inicial, que a literatura aponta como atrativos do JavaScript (Shrestha et al., 2022).

A análise dos dados coletados será realizada de forma quantitativa e comparativa. As respostas dos dois formulários serão exportadas e tabuladas em planilhas eletrônicas. Serão aplicadas técnicas de estatística descritiva, como cálculo de médias, medianas e distribuições de frequência, para sumarizar as percepções de cada grupo. O foco da análise será comparar as respostas dos profissionais experientes com as dos iniciantes, identificando diferenças e semelhanças estatisticamente relevantes nas suas percepções sobre os benefícios e desafios do JavaScript e do TypeScript. Essa comparação direta permitirá validar ou refutar as hipóteses levantadas na fundamentação teórica, oferecendo uma contribuição empírica sobre como o nível de experiência influencia a preferência e a avaliação de tecnologias no desenvolvimento front-end.

4 Análise e Discussão dos Resultados

Este capítulo apresenta a consolidação, análise e discussão aprofundada dos dados empíricos obtidos a partir da aplicação dos questionários estruturados a desenvolvedores de software front-end. O objetivo central é confrontar as hipóteses teóricas estabelecidas na literatura com as percepções reais da prática profissional, identificando como fatores como tempo de atuação na área e vivência em projetos de grande escala moldam a avaliação de JavaScript e TypeScript.

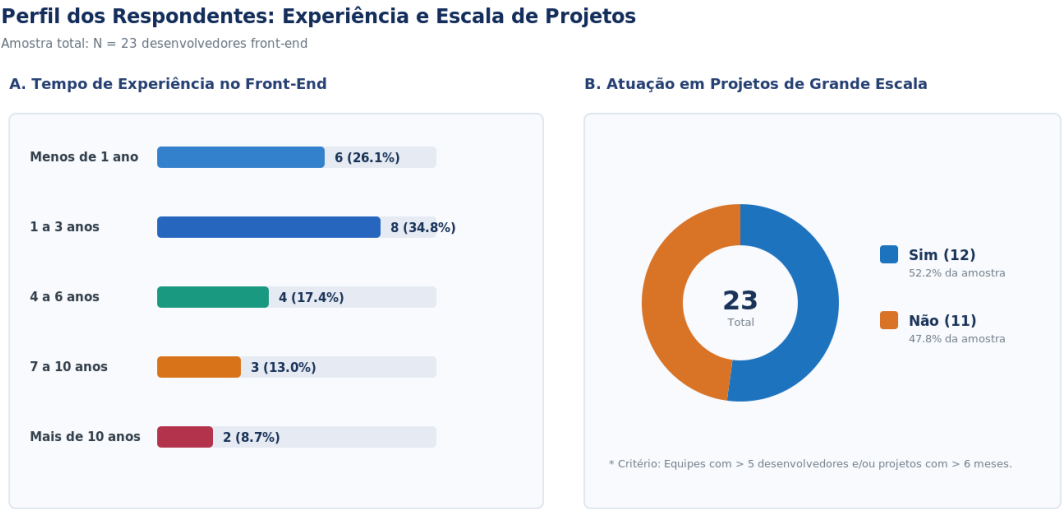
A análise está estruturada em seis seções: inicialmente, caracteriza-se o perfil demográfico da amostra; em seguida, examinam-se as dimensões de produtividade e agilidade de desenvolvimento (Bloco A); na sequência, analisam-se os aspectos de qualidade, manutenibilidade e prevenção de falhas (Bloco B); posteriormente, avalia-se o ecossistema, o suporte de ferramentas e os desafios de migração (Bloco C); discutem-se então as preferências pessoais, o nível de confiança e as perspectivas futuras (Bloco D); por fim, realiza-se uma categorização temática dos relatos qualitativos abertos e uma síntese de confronto com os referenciais teóricos.

4.1 Caracterização Demográfica dos Participantes

A pesquisa contou com a participação de um total de $N = 23$ desenvolvedores. A segmentação dos respondentes seguiu os parâmetros metodológicos delineados no Capítulo 3, permitindo agrupar a amostra de acordo com o tempo de experiência no desenvolvimento front-end e a vivência em projetos de grande escala (definidos como iniciativas com mais de 5 desenvolvedores e/ou duração superior a 6 meses).

A distribuição da amostra quanto ao tempo de experiência e participação em projetos corporativos de grande porte está ilustrada na Figura 1.

Figura 1 – Perfil dos respondentes por tempo de experiência e participação em grandes projetos



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Fonte: Dados da pesquisa aplicada pelo autor (2025).

Conforme evidencia a Figura 1(A), os respondentes distribuem-se em diferentes estágios de maturidade na carreira:

- **Menos de 1 ano:** 6 participantes (26,1%);
- **1 a 3 anos:** 8 participantes (34,8%);
- **4 a 6 anos:** 4 participantes (17,4%);
- **7 a 10 anos:** 3 participantes (13,0%);
- **Mais de 10 anos:** 2 participantes (8,7%).

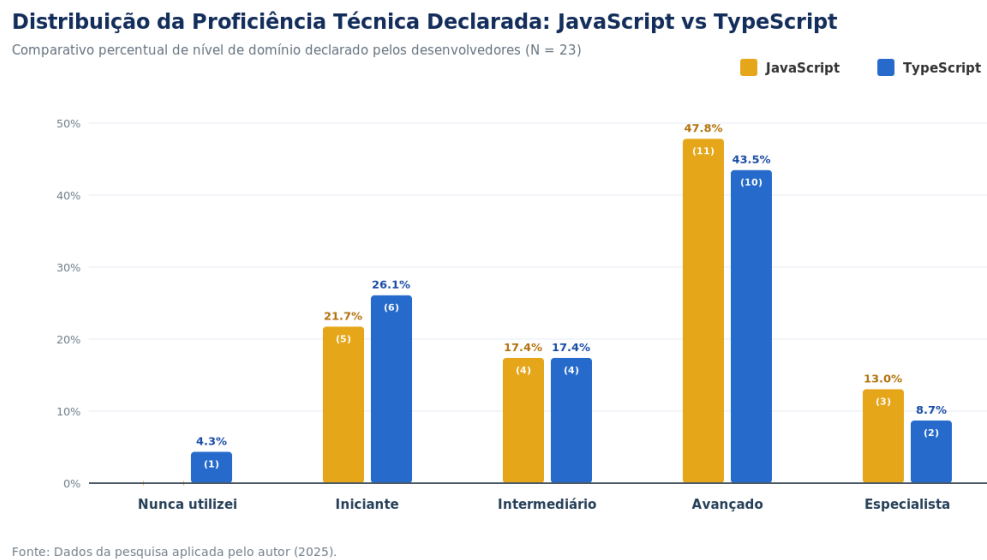
Para fins de análise comparativa neste estudo, foram estabelecidos dois grupos principais:

1. **Grupo de Desenvolvedores Iniciantes e Intermediários** ($N = 14$, **60,9%**): composto por profissionais e estudantes em fase de formação com até 3 anos de experiência em desenvolvimento front-end.
2. **Grupo de Profissionais Experientes** ($N = 9$, **39,1%**): formado por desenvolvedores com 4 ou mais anos de experiência comprovada no mercado de tecnologia, incluindo seniores e especialistas técnicos.

Adicionalmente, a Figura 1(B) demonstra que 12 participantes (52,2%) possuem experiência sólida em projetos de grande escala, enquanto 11 participantes (47,8%) atuaram predominantemente em projetos individuais, de menor porte ou em contexto estritamente acadêmico. Esse equilíbrio amostral é particularmente valioso para avaliar se as dores e os benefícios apreendidos pela adoção do TypeScript dependem da escala organizacional do software.

A Figura 2 apresenta a comparação entre os níveis declarados de proficiência técnica em JavaScript e em TypeScript.

Figura 2 – Comparativo de proficiência técnica autoavaliada em JavaScript e TypeScript



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Os dados revelam que o domínio de JavaScript é consolidado na totalidade da amostra, com 60,8% dos participantes posicionando-se nos níveis *Avançado* (47,8%) ou *Especialista* (13,0%), e nenhum respondente declarando desconhecimento da linguagem. Em relação ao TypeScript, observa-se uma distribuição similar no topo da pirâmide (52,2% entre avançados e especialistas), porém com uma fração mais expressiva de iniciantes (26,1%) e um participante (4,3%) que declarou nunca ter utilizado a ferramenta. Esse panorama reflete com precisão a dinâmica descrita por (Mororó, 2024), em que o JavaScript atua como fundação prévia obrigatória para a assimilação subsequente do TypeScript.

4.2 Produtividade e Desenvolvimento Ágil (Bloco A)

O Bloco A do questionário investigou a percepção dos participantes acerca da velocidade de desenvolvimento, facilidade de prototipagem, curva de aprendizado e detecção de

falhas. As respostas foram mensuradas por meio de uma escala Likert de 5 pontos (variando de 1 = Discordo totalmente a 5 = Concordo totalmente).

A Tabela 1 sumariza as métricas estatísticas descritivas (médias, medianas e desvios-padrão) para o conjunto geral da amostra e a desagregação comparativa entre desenvolvedores iniciantes e experientes.

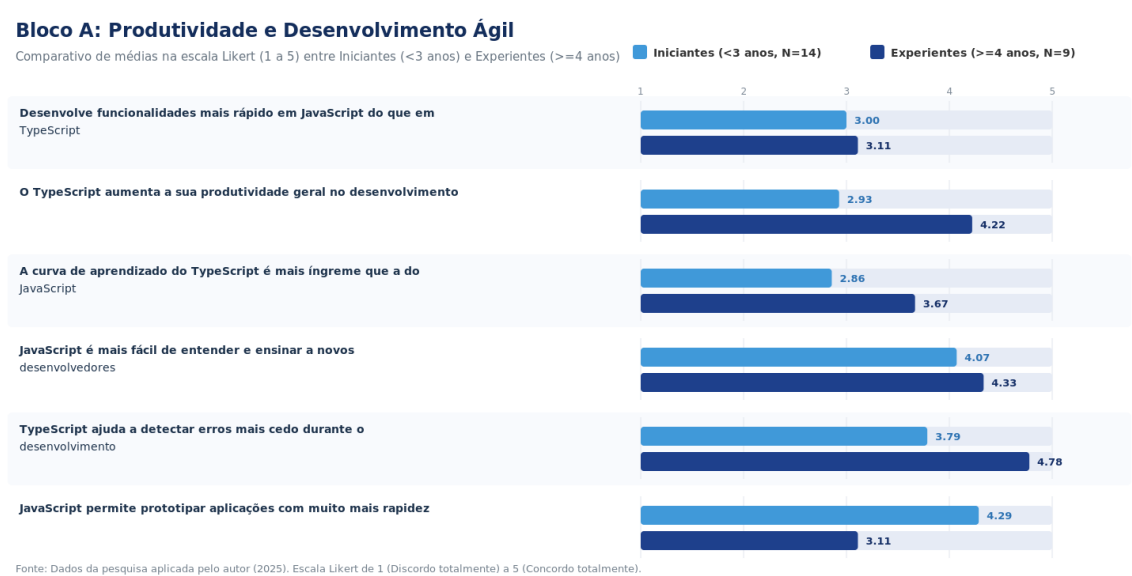
Tabela 1 – Estatísticas descritivas do Bloco A: Produtividade e Desenvolvimento

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
A1. Desenvolve mais rápido em JS do que em TS	3,04	3,0	1,22	3,00	3,11	+0,11
A2. O TS aumenta a produtividade no desenvolvimento	3,43	4,0	1,41	2,93	4,22	+1,29
A3. A curva de aprendizado do TS é mais íngreme	3,17	3,0	1,07	2,86	3,67	+0,81
A4. JS é mais fácil de entender e ensinar a novos	4,17	4,0	0,89	4,07	4,33	+0,26
A5. O TS ajuda a detectar erros mais cedo	4,17	5,0	1,19	3,79	4,78	+0,99
A6. O JS permite prototipar aplicações mais rápido	3,83	4,0	1,23	4,29	3,11	-1,18

Fonte: Dados da pesquisa aplicada pelo autor (2025).

A representação gráfica comparativa dessas percepções é ilustrada na Figura 3.

Figura 3 – Comparativo de médias no Bloco A entre Iniciantes e Experientes



Fonte: Dados da pesquisa aplicada pelo autor (2025).

A análise dos resultados do Bloco A evidencia conclusões fundamentais sobre o impacto da maturidade profissional na percepção de produtividade:

1. **A Divergência Crítica de Produtividade** ($\Delta = +1, 29$): Enquanto os desenvolvedores iniciantes mantêm uma postura neutra ou reticente quanto aos ganhos de produtividade proporcionados pelo TypeScript (média de 2, 93), os profissionais experientes manifestam forte concordância de que a tecnologia eleva expressivamente o seu rendimento (média de 4, 22). Para o desenvolvedor experiente, a sobrecarga de digitar assinaturas de tipos é amplamente compensada pelo suporte inteligente de autocompletar da IDE e pela eliminação de ciclos manuais de teste e depuração.
2. **Prototipagem Rápida e Flexibilidade** ($\Delta = -1, 18$): O grupo de iniciantes concorda enfaticamente que o JavaScript puro possibilita criar protótipos de forma substancialmente mais veloz (média de 4, 29). Em contrapartida, os desenvolvedores seniores atribuem nota significativamente menor a essa afirmação (média de 3, 11). Isso decorre do fato de que, em estágios iniciais de aprendizado, a rigidez do compilador do TypeScript é vivenciada como uma barreira burocrática, ao passo que a ausência de amarras no JavaScript confere uma sensação imediata de liberdade e rapidez.
3. **Deteção Precoce de Erros** ($\Delta = +0, 99$): Ambos os grupos reconhecem o papel do TypeScript na identificação de defeitos antes do envio para produção, contudo a adesão dos seniores atinge patamares quase unânimes (média de 4, 78 e mediana 5, 0). Esse resultado corrobora diretamente os achados de (Richards; Nardelli; Vitek, 2017) e (Makimov, 2022), indicando que a capacidade de antecipar inconsistências durante a fase de codificação constitui o pilar central de valor percebido por quem atua em ambientes corporativos contínuos.
4. **Facilidade Pedagógica**: Tanto iniciantes (4, 07) quanto experientes (4, 33) concordam amplamente que o JavaScript é uma linguagem didaticamente mais acessível para a introdução de novos programadores à web, reflexo direto de sua sintaxe não vinculada à tipagem estrita prévia.

—

4.3 Qualidade do Código, Manutenibilidade e Prevenção de Falhas (Bloco B)

O Bloco B concentrou-se na análise das características intrínsecas da base de código: legibilidade, facilidade de manutenção em longo prazo, propensão à geração de defeitos, verbosidade e padronização arquitetural em equipes multidisciplinares.

Os dados estatísticos consolidados do Bloco B estão descritos na Tabela 2.

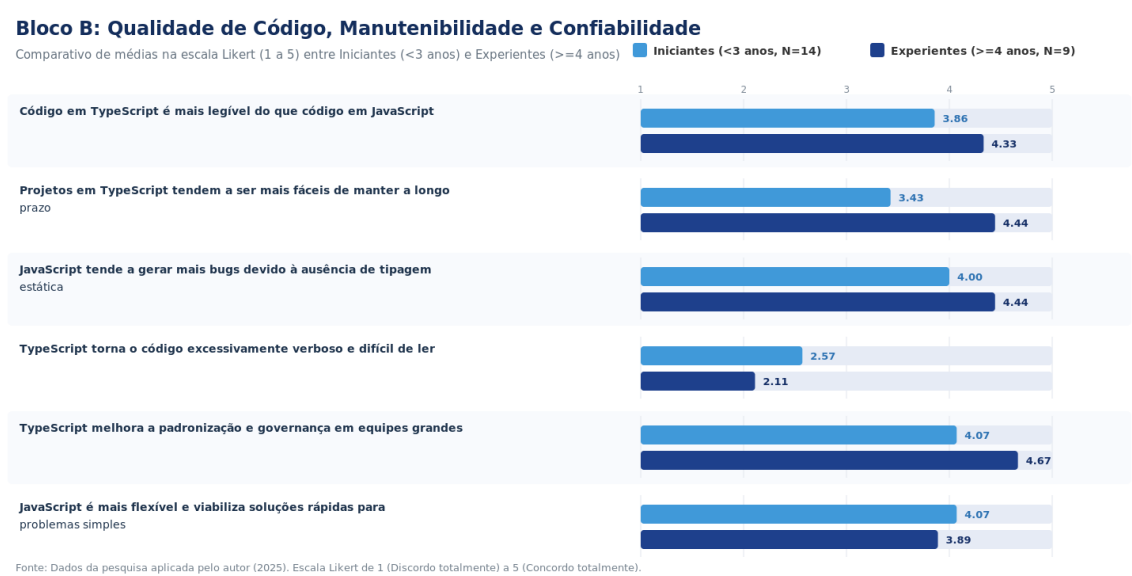
Tabela 2 – Estatísticas descritivas do Bloco B: Qualidade e Manutenção do Código

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
B1. Código em TS é mais legível que em JS	4,04	5,0	1,33	3,86	4,33	+0,47
B2. Projetos em TS são mais fáceis de manter a longo prazo	3,83	4,0	1,34	3,43	4,44	+1,01
B3. JS gera mais bugs por ausência de tipagem	4,17	5,0	1,15	4,00	4,44	+0,44
B4. O TS torna o código excessivamente verboso e difícil	2,39	2,0	1,16	2,57	2,11	-0,46
B5. TS melhora a padronização em equipes grandes	4,30	5,0	0,88	4,07	4,67	+0,60
B6. JS facilita soluções rápidas para problemas simples	4,00	4,0	1,04	4,07	3,89	-0,18

Fonte: Dados da pesquisa aplicada pelo autor (2025).

A Figura 4 ilustra graficamente o comportamento comparativo das variáveis avaliadas no Bloco B.

Figura 4 – Comparativo de médias no Bloco B entre Iniciantes e Experientes



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Os resultados do Bloco B reforçam aspectos cruciais da literatura de Engenharia de Software:

1. **Manutenibilidade em Longo Prazo ($\Delta = +1,01$):** A assertiva de que bases de código estruturadas em TypeScript são mais sustentáveis e manuteníveis ao longo do tempo recebeu avaliação destacada dos desenvolvedores seniores (média de 4,44). Ao segmentar a amostra por participação em grandes projetos, essa média se consolida em 4,42 (contra

3,18 entre aqueles que nunca vivenciaram projetos extensos). Isso demonstra empiricamente que a percepção do retorno sobre o investimento (ROI) da tipagem estática cresce exponencialmente à medida que os sistemas amadurecem e sofrem contínuas manutenções evolutivas.

2. **Origem de Bugs por Ausência de Tipagem Estática:** A premissa de que a tipagem dinâmica do JavaScript é um vetor prolífico de defeitos em tempo de execução obteve alta concordância em ambos os grupos (4,00 entre iniciantes e 4,44 entre experientes; média geral de 4,17). Tal constatação alinha-se às observações de (Bogner; Wagner, 2022) e (George, 2020), ratificando que a manipulação de estruturas de dados sem contratos explícitos gera vulnerabilidades funcionais recorrentes.
3. **Desmistificação da Sobrecarga de Verbosidade:** A hipótese de que o TypeScript tornaria o código excessivamente prolixo ou ilegível foi amplamente rejeitada pelos participantes (média global de 2,39; 2,11 entre experientes e 2,00 entre os atuantes em grandes projetos). Em contrapartida, a legibilidade do código em TypeScript foi avaliada positivamente (4,04 geral e 4,33 para experientes), indicando que os desenvolvedores encaram as anotações de tipo não como ruído, mas como documentação viva acoplada ao código-fonte.
4. **Governança em Equipes Grandes:** O item que associa o TypeScript ao aprimoramento da padronização e consistência em times numerosos obteve a maior média de todo o bloco (4,30 geral e 4,67 entre experientes, com mediana 5,0). Esse dado corrobora a tese de que os tipos funcionam como um contrato formal de comunicação interequipes.

—

4.4 Compatibilidade, Ferramentas e Desafios de Migração (Bloco C)

O Bloco C investigou a relação do TypeScript com o ferramental moderno de desenvolvimento (IDEs, linters, formatadores), o ecossistema de frameworks de ponta (React, Angular, Vue) e os custos e dificuldades de integrar ou migrar sistemas concebidos originalmente em JavaScript.

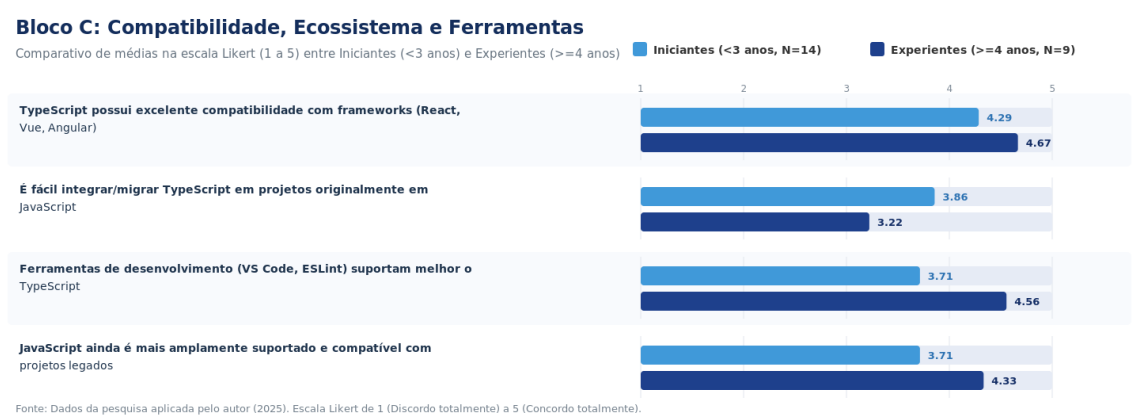
A Tabela 3 e a Figura 5 sintetizam as respostas obtidas.

Tabela 3 – Estatísticas descritivas do Bloco C: Compatibilidade e Ferramentas

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
C1. TS possui excelente suporte em bibliotecas modernas	4,43	5,0	0,66	4,29	4,67	+0,38
C2. É fácil migrar/integrar TS em projetos legados JS	3,61	4,0	1,12	3,86	3,22	-0,64
C3. Ferramentas (IDEs/Linters) suportam melhor o TS	4,04	4,0	0,93	3,71	4,56	+0,85
C4. JS ainda é mais amplamente suportado	3,96	4,0	1,07	3,71	4,33	+0,62

Fonte: Dados da pesquisa aplicada pelo autor (2025).

Figura 5 – Comparativo de médias no Bloco C entre Iniciantes e Experientes



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Dentre os principais achados do Bloco C, destacam-se:

- 1. Simbiose com o Ecossistema e Ferramental ($\Delta = +0,85$):** O suporte proporcionado por ambientes de desenvolvimento integrados como o Visual Studio Code, somado a extensões de análise estática (ESLint, Prettier), é avaliado com grau extremo de satisfação pelos desenvolvedores seniores (média de 4,56). O TypeScript transforma o editor de texto em uma ferramenta analítica completa de refatoração, navegação semântica e auto-completar contextual.
- 2. O Paradoxo da Migração de Projetos Legados ($\Delta = -0,64$):** Observa-se uma interessante inversão de perspectiva entre os grupos: os desenvolvedores iniciantes avaliam o processo de migração de JavaScript para TypeScript como relativamente fácil (média de 3,86), enquanto os profissionais experientes expressam maior cautela e ceticismo (média de 3,22). Essa divergência fundamenta-se na vivência prática: os seniores já enfrentaram os gargalos reais de migrar bases extensas, incluindo a necessidade de tipar pacotes de terceiros obsoletos, gerenciar conflitos de compilação no `tsconfig.json` e refatorar

padrões altamente dinâmicos de metaprogramação herdados do ecossistema JavaScript puro.

3. **Universalidade dos Frameworks:** A compatibilidade do TypeScript com frameworks front-end predominantes (React, Vue, Angular) recebeu uma das maiores médias gerais da pesquisa (4,43 e desvio-padrão de apenas 0,66), comprovando que a indústria estabeleceu o TypeScript como padrão de primeira classe na arquitetura dessas ferramentas.

—

4.5 Preferências Pessoais, Confiança e Perspectivas Futuras (Bloco D)

O Bloco D aferiu o vínculo atitudinal e emocional dos desenvolvedores em relação às duas linguagens, mensurando aspectos como nível de confiança psicológica ao codificar, preferência para novos projetos (*greenfield*) e a projeção de relevância futura no mercado.

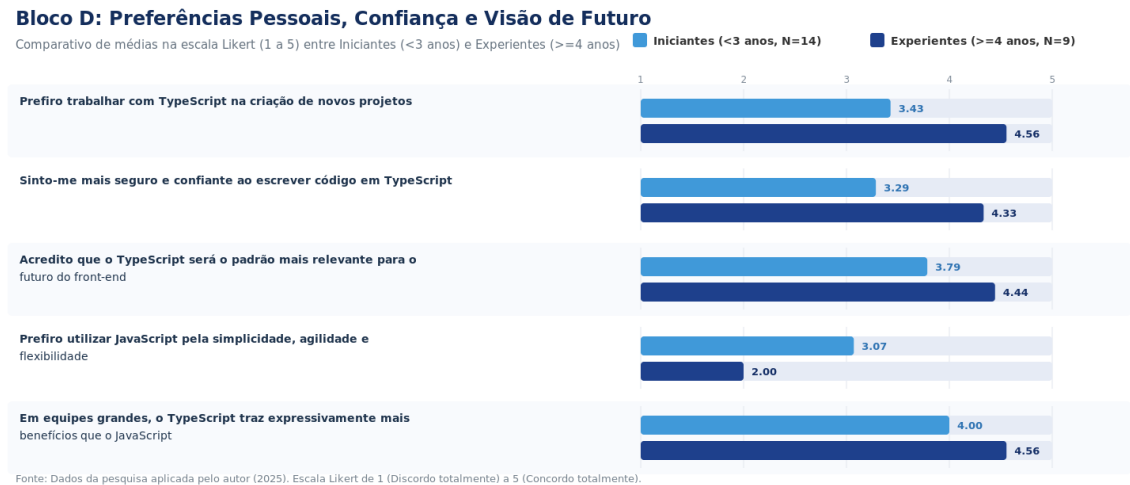
A Tabela 4 e a Figura 6 detalham as médias apuradas.

Tabela 4 – Estatísticas descritivas do Bloco D: Preferências e Perspectivas Pessoais

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
D1. Prefiro trabalhar com TS em novos projetos	3,87	4,0	1,36	3,43	4,56	+1,13
D2. Sinto-me mais seguro e confiante com TS	3,70	4,0	1,33	3,29	4,33	+1,04
D3. Acredito que o TS será o futuro do front-end	4,04	4,0	1,11	3,79	4,44	+0,65
D4. Prefiro JS pela simplicidade e flexibilidade	2,65	3,0	1,50	3,07	2,00	-1,07
D5. Em equipes grandes, TS traz mais benefícios	4,22	5,0	1,00	4,00	4,56	+0,56

Fonte: Dados da pesquisa aplicada pelo autor (2025).

Figura 6 – Comparativo de médias no Bloco D entre Iniciantes e Experientes



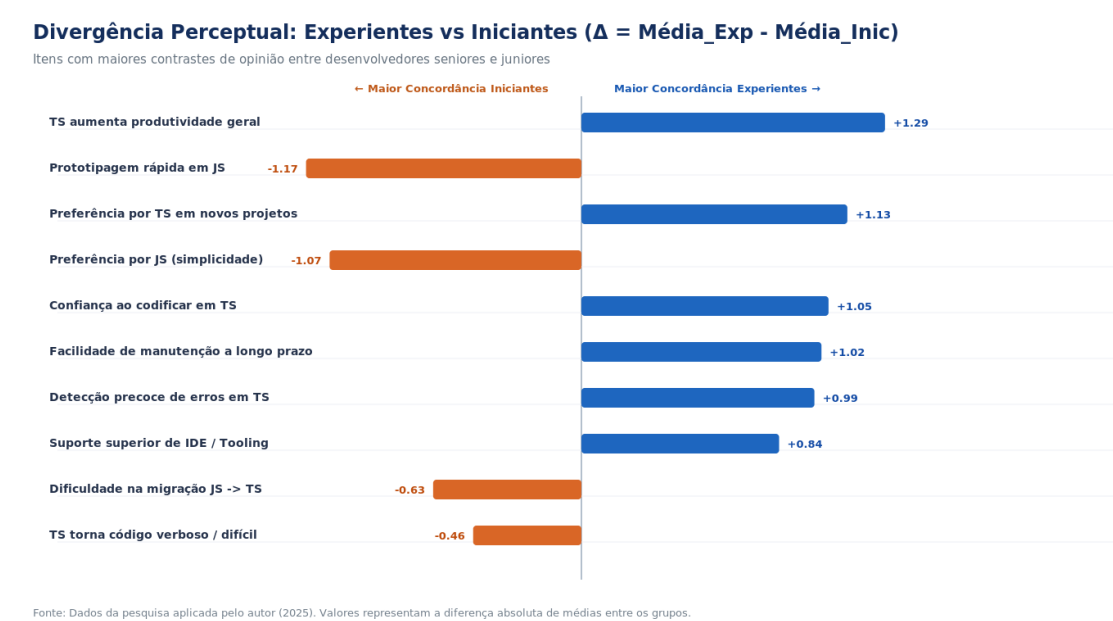
Fonte: Dados da pesquisa aplicada pelo autor (2025).

A dimensão atitudinal traz revelações expressivas:

- Segurança Psicológica e Confiança Técnica ($\Delta = +1,04$):** Os desenvolvedores experientes expressam um nível significativamente mais elevado de segurança ao codificar em TypeScript (4,33 contra 3,29 dos iniciantes). A validação estática proporciona uma rede de proteção contra erros triviais de sintaxe e tipo, liberando a carga cognitiva do desenvolvedor para focar na lógica de negócios da aplicação.
- Preferência para Novos Projetos ($\Delta = +1,13$):** Diante da oportunidade de iniciar um projeto a partir do zero, os profissionais seniores manifestam preferência quase hegemônica pelo TypeScript (4,56). Por outro lado, a preferência pelo JavaScript devido à sua simplicidade cai drasticamente com a experiência (2,00 entre seniores e 1,83 entre atuantes em grandes projetos, contra 3,07 e 3,55 entre iniciantes e não-participantes de grandes projetos).
- Projeção de Relevância Futura:** Há um consenso expressivo na comunidade pesquisada (média geral de 4,04) de que o TypeScript continuará expandindo sua relevância e consolidando-se como o padrão dominante na engenharia de front-end.

A Figura 7 sintetiza as principais divergências perceptuais entre os dois grupos, ordenadas pela magnitude do delta ($\Delta = \text{Média}_{\text{Experientes}} - \text{Média}_{\text{Iniciantes}}$).

Figura 7 – Divergência perceptual consolidada entre Experientes e Iniciantes (Δ de Médias)



Fonte: Dados da pesquisa aplicada pelo autor (2025).

4.6 Análise Qualitativa dos Relatos Práticos da Indústria

A questão aberta final do formulário solicitou aos participantes que relatassem, de forma contextualizada, experiências práticas onde o uso de TypeScript trouxe vantagens tangíveis ou desvantagens operacionais em relação ao JavaScript puro.

A análise temático-categorial desses depoimentos permitiu consolidar quatro eixos fundamentais que explicam a dinâmica real de adoção na indústria de software:

4.6.1 Eixo 1: Blindagem de Contratos de Dados e Integração de APIs

O benefício mais recorrente citado pelos profissionais foi o estabelecimento de interfaces e contratos estritos na comunicação entre camadas da aplicação e consumo de serviços REST/GraphQL.

Um desenvolvedor com mais de 10 anos de experiência (Respondente 13) relatou:

“Em projetos Node.js de médio e grande porte, o TypeScript trouxe ganhos significativos de qualidade ao permitir tipagem estática, melhor documentação do código e detecção precoce de erros durante o desenvolvimento. Em especial, em integra-

ções entre múltiplos serviços e APIs, a tipagem ajudou a reduzir falhas de contrato entre componentes e tornou refatorações mais seguras.” (Respondente 13, Mais de 10 anos de experiência).

De forma complementar, o Respondente 1 destacou a aplicação em fluxos com regras de negócios críticas:

“Em projetos como o meu livrinho mágico e o portal da motopel o typescript ajudou porque deixou os dados mais seguros por exemplo: ao lidar com usuários, páginas do livro imagens e respostas da api ficaram mais fáceis de evitar erros, que no javascript só apareceriam com o sistema rodando.” (Respondente 1, 1 a 3 anos de experiência).

4.6.2 Eixo 2: Refatoração em Larga Escala e Propagação de Modificações

A segurança durante processos de refatoração constitui uma das maiores vantagens operacionais do TypeScript. A existência de tipos algébricos e uniões discriminadas permite que o compilador sinalize automaticamente todos os trechos do sistema que necessitam de adaptação após uma alteração de esquema.

O Respondente 5 (7 a 10 anos de experiência) ilustrou esse mecanismo de forma precisa:

“A atualização de um union type no typescript foi suficiente para gerar erros em todos os pontos que precisavam de atualização no código. Com javascript, seria necessária uma busca manual dos pontos de atualização, já que os erros não seriam manifestados.” (Respondente 5, Especialista em JS/TS).

Essa garantia mecânica elimina o risco de regressões silenciosas decorrentes de alterações em modelos de dados centrais, aspecto que frequentemente inviabiliza refatorações ambiciosas em projetos legados em JavaScript.

4.6.3 Eixo 3: Complexidade Inicial e Fricção com o Ecossistema Externo

Em contrapartida às vantagens de longo prazo, os participantes foram unânimes em apontar os custos de partida associados ao TypeScript: aumento do tempo de configuração de ambiente (*toolchain*), lentidão inicial para modelar interfaces complexas e atrito com bibliotecas que não disponibilizam definições de tipos adequadas.

O Respondente 12 pontuou essa relação de *trade-off*:

“Em um projeto utilizando Vue.js com Nuxt 3 e TypeScript, trabalhei na implementação de formulários complexos [...]. Houve desvantagens no aumento da complexidade inicial, principalmente ao lidar com tipagens mais avançadas e integração com bibliotecas externas que nem sempre possuem tipagem bem definida, exigindo ajustes manuais ou uso pontual de any. Mesmo assim, o ganho em segurança e escalabilidade compensou o custo inicial.” (Respondente 12, 4 a 6 anos de experiência).

Esse relato corrobora o achado quantitativo de que desenvolvedores seniores reconhecem a dificuldade de migração e configuração inicial, embora julguem que os benefícios posteriores em manutenibilidade compensem amplamente o custo despendido.

4.6.4 Eixo 4: A Fronteira entre Compilação e Execução (Runtime vs Compile-time)

Um aspecto técnico de extrema relevância, apontado com lucidez pelo Respondente 6, diz respeito à natureza da transpilação do TypeScript e seus limites de proteção:

“TypeScript por apresentar tipagem deixa o código muito mais legível e organizado, além de prevenir erros [...]. Porém o TS ele só vai validar e ver errors durante o desenvolvimento, em runtime ele ainda vai aceitar uma String em uma variável declarada como Number, porque por baixo ele vai rodar em JS. então você tem que ficar atento para validar e tratar os dados que está mexendo.” (Respondente 6, 1 a 3 anos de experiência).

Essa observação adverte para a falsa sensação de segurança que o TypeScript pode induzir caso o desenvolvedor presuma que a verificação estática elimina a necessidade de validações em tempo de execução para dados externos (como entradas de formulários e respostas HTTP não confiáveis).

—

4.7 Confronto Teórico-Empírico e Síntese da Discussão

O Quadro 1 estabelece a síntese integrativa confrontando as premissas formuladas na literatura com os resultados empíricos apurados nesta pesquisa.

Quadro 1 – Confronto entre a literatura acadêmica e as evidências empíricas da pesquisa

Dimensão Teórica	Postulado da Literatura	Evidência Empírica da Pesquisa
Prevenção de Bugs e Robustez	O sistema de tipos estáticos previne falhas silenciosas e erros de tipo antes da execução (Richards; Nardelli; Vitek, 2017).	Validado. Média global de 4,17 concordando que JS gera mais bugs; 4,78 entre experientes afirmando detecção precoce no TS.
Curva de Aprendizado e Transferência	O conhecimento prévio em JS gera atrito conceitual e hábitos que dificultam a assimilação de TS (Shrestha <i>et al.</i> , 2022).	Validado. Iniciantes valorizam a flexibilidade do JS (4,29 em prototipagem) e sentem sobrecarga inicial com tipagem estrita de TS.
Qualidade de Código vs Complexidade	Aplicações em TS apresentam melhor arquitetura, documentação viva e menor complexidade cognitiva (Bogner; Wagner, 2022).	Validado. Alta avaliação de legibilidade (4,04), padronização de equipes (4,30) e rejeição da alegação de verbosidade excessiva (2,39).
Evolução e Tendência de Mercado	Migração massiva da indústria impulsionada por suporte de frameworks e tooling (George, 2020; Stack Overflow, 2023).	Validado. Suporte de ecossistema (4,43), tooling/IDEs (4,56 para experientes) e ampla projeção de relevância futura (4,04).

Fonte: Elaborado pelo autor (2025).

Em suma, os resultados revelam que a escolha entre JavaScript e TypeScript não é uma mera disputa estilística, mas sim uma decisão arquitetural fortemente condicionada pelo horizonte temporal do projeto e pela maturidade da equipe. Enquanto o JavaScript reina na prototipagem rápida e na introdução pedagógica, o TypeScript consolida-se como o padrão indispensável para a viabilidade, confiabilidade e sustentabilidade de sistemas web modernos de grande escala.

5 Considerações Finais

Este trabalho teve como objetivo central investigar as percepções e os fatores que influenciam a adoção de JavaScript e TypeScript no desenvolvimento front-end, analisando comparativamente as diferenças entre desenvolvedores iniciantes e profissionais experientes. A coexistência dessas duas linguagens constitui um dos temas mais relevantes da engenharia de software contemporânea, moldando decisões arquiteturais, estratégias de contratação e práticas de governança de código em organizações de tecnologia de todos os portes.

Por meio de uma abordagem metodológica quantitativa e qualitativa baseada em levantamento de dados (*survey*), foi possível coletar e analisar as perspectivas de 23 desenvolvedores com diferentes níveis de maturidade profissional e vivência em projetos de variadas escalas. A seguir, apresentam-se as principais conclusões obtidas, as contribuições práticas do estudo, suas limitações e as recomendações para trabalhos futuros.

5.1 Síntese dos Resultados e Atendimento aos Objetivos

Os resultados empíricos obtidos permitiram responder integralmente aos objetivos específicos delineados na introdução deste trabalho:

1. **Análise comparativa das características técnicas:** A fundamentação teórica e a pesquisa empírica evidenciaram que o sistema de tipagem estática e opcional do TypeScript representa uma evolução estrutural sobre a tipagem dinâmica do JavaScript. Embora o JavaScript ofereça agilidade imediata e ausência de etapas formais de compilação, o TypeScript desloca a detecção de incompatibilidades de tipos do tempo de execução para o tempo de compilação, prevenindo falhas silenciosas em produção.
2. **Identificação dos benefícios e desafios do TypeScript:** Os dados confirmaram que os principais benefícios do TypeScript residem na expressiva manutenibilidade a longo prazo (média de 4,44 entre experientes), na prevenção de bugs (média global de 4,17), na padronização arquitetural em equipes grandes (média de 4,30) e na segurança durante refatorações críticas. Em contrapartida, os principais desafios identificados consistem no custo inicial de configuração de ambiente, no atrito com bibliotecas de terceiros desprovidas de tipagem adequada e na necessidade de disciplina técnica contínua.
3. **Avaliação do impacto do nível de experiência:** Constatou-se uma nítida divergência perceptual condicionada pelo tempo de carreira e vivência em projetos de grande escala. Enquanto iniciantes valorizam a flexibilidade e a prototipagem rápida do JavaScript (média de 4,29), os profissionais seniores experimentam um salto líquido de produtividade ao

utilizar TypeScript (média de 4, 22, contra 2, 93 dos iniciantes), manifestando preferência hegemônica (4, 56) pela sua utilização em novos projetos corporativos.

5.2 Contribuições e Recomendações Práticas

A partir dos achados desta pesquisa, formulam-se diretrizes aplicadas destinadas tanto à gestão de projetos de software quanto à formação acadêmica:

5.2.1 Diretrizes para a Indústria e Gestores de Tecnologia

- **Projetos Corporativos e de Longo Prazo:** Recomenda-se a adoção mandatória de TypeScript para aplicações empresariais, sistemas colaborativos e bases de código com previsão de sustentação contínua, uma vez que a tipagem atua como documentação autoexecutável e barreira contra regressões.
- **Prototipagem e MVPs:** O uso de JavaScript puro permanece altamente justificável em estágios exploratórios de produtos (*Minimum Viable Products*), validações conceituais imediatas ou scripts utilitários onde a velocidade inicial sobrepõe-se à rigidez estrutural.
- **Estratégias de Migração Gradual:** Equipes que planejam migrar sistemas legados de JavaScript para TypeScript devem prever um investimento deliberado em capacitação inicial e adotar estratégias incrementais (ativando modos permissivos antes de aplicar regras estritas de tipagem), evitando o uso indiscriminado do tipo `any`.

5.2.2 Diretrizes para o Ensino Acadêmico de Computação

- **Progressão Pedagógica:** Cursos de graduação e técnicos em computação e sistemas de informação devem manter o JavaScript no ciclo básico introdutório, dada sua menor barreira de entrada e facilidade didática (4, 17 geral).
- **Maturidade em Engenharia de Software:** Deve-se introduzir o TypeScript nas disciplinas avançadas de desenvolvimento web e arquitetura de software, preparando os futuros profissionais para as exigências reais do mercado de trabalho corporativo.

5.3 Limitações da Pesquisa

Como em qualquer estudo empírico, algumas limitações metodológicas devem ser consideradas na interpretação dos resultados:

1. **Tamanho da Amostra:** A pesquisa baseou-se em uma amostra de $N = 23$ participantes, selecionada por conveniência em comunidades de tecnologia e no ambiente acadêmico do IFBA. Embora a amostra seja diversificada e tenha possibilitado análises comparativas consistentes, generalizações estatísticas para a totalidade da comunidade global de desenvolvedores devem ser realizadas com parcimônia.
2. **Natureza Autorreferida:** Os dados fundamentam-se nas percepções e autoavaliações dos próprios desenvolvedores, refletindo experiências subjetivas que podem ser influenciadas por preferências pessoais e tendências de mercado.

5.4 Sugestões para Trabalhos Futuros

Para dar continuidade e aprofundamento à linha de pesquisa explorada neste trabalho de conclusão de curso, sugerem-se as seguintes investigações:

1. **Estudo de Mineração de Repositórios:** Realizar uma análise quantitativa automatizada em larga escala de repositórios públicos (GitHub/GitLab) de empresas brasileiras, extraíndo métricas concretas de densidade de bugs, complexidade ciclomática e rotatividade de código (*code churn*) em projetos JS e TS.
2. **Impacto de Assistentes de Inteligência Artificial:** Investigar como o surgimento de ferramentas de inteligência artificial generativa e autocompletar inteligente (como GitHub Copilot e modelos baseados em LLM) impacta a curva de aprendizado de linguagens fortemente tipadas como o TypeScript.
3. **Estudo Longitudinal de Migração:** Acompanhar equipes de desenvolvimento durante o ciclo completo de migração de uma base legada em JavaScript para TypeScript, mensurando o tempo despendido, os tipos de erros encontrados e o impacto na velocidade de entrega (*lead time*).

Em conclusão, o TypeScript consolidou-se como um pilar fundamental da engenharia de front-end moderna, demonstrando que a imposição de tipos estáticos, longe de ser um obstáculo burocrático, constitui uma poderosa ferramenta de produtividade, governança e confiabilidade para a construção de sistemas sustentáveis.

Quadro 5 – Cronograma de atividades do TCC

Etapas	Mar/25	Abr/25	Mai/25	Jun/25	Jul/25	Ago/25	Set/25	Out/25	Nov/25	Dez/25	Jan/26	Fev/26	Mar/26
Escolha do tema	X												
Levantamento bibliográfico	X	X											
Definição do problema e objetivos		X	X										
Introdução			X	X									
Fundamentação Teórica				X	X								
Metodologia					X	X	X	X					
Foirmulario								X	X				
Conclusão										X	X		
Revisão geral do trabalho												X	X
Preparação da apresentação/defesa												X	X

Fonte: Elaborado pelo autor, 2025.

Referências

- BOGNER, Justus; WAGNER, Stefan. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. *In: PROCEEDINGS of the 19th International Conference on Mining Software Repositories*. 2022. p. 552–563. DOI: [10.1145/3524842.3528454](https://doi.org/10.1145/3524842.3528454). Disponível em: <https://dl.acm.org/doi/abs/10.1145/3524842.3528454>.
- CRISTIANI, Fernando; THIEMANN, Peter. Generation of TypeScript declaration files from JavaScript code. *In: PROCEEDINGS of the 18th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*. 2021. p. 97–112. DOI: [10.1145/3475738.3480941](https://doi.org/10.1145/3475738.3480941).
- FELDTTHAUS, Asger; MØLLER, Anders. Checking correctness of TypeScript interfaces for JavaScript libraries. *In: PROCEEDINGS of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications*. 2014. p. 1–16. DOI: [10.1145/2660193.2660215](https://doi.org/10.1145/2660193.2660215).
- GEORGE, Juliana. Adoption trends: Why developers are switching from JavaScript to TypeScript. **ResearchGate**, 2020. Disponível em: https://www.researchgate.net/publication/389688708_Adoption_Trends_Why_Developers_Are_Switching_from_JavaScript_to_TypeScript.
- HOEFLICH, Joshua; FINDLER, Robert Bruce; SERRANO, Manuel. Highly illogical, Kirk: Spotting type mismatches in the large despite broken contracts, unsound types, and too many linters. **Proceedings of the ACM on Programming Languages**, v. 6, OOPSLA2, p. 479–504, 2022. DOI: [10.1145/3563305](https://doi.org/10.1145/3563305).
- JESSE, Kevin; DEVANBU, Premkumar T. ManyTypes4TypeScript: A comprehensive TypeScript dataset for sequence-based type inference. *In: PROCEEDINGS of the 19th International Conference on Mining Software Repositories*. 2022. p. 294–298. DOI: [10.1145/3524842.3528507](https://doi.org/10.1145/3524842.3528507).
- JESSE, Kevin; DEVANBU, Premkumar T.; SAWANT, Anand. Learning to predict user-defined types. **IEEE Transactions on Software Engineering**, v. 49, n. 4, p. 1508–1522, 2023. DOI: [10.1109/TSE.2022.3178945](https://doi.org/10.1109/TSE.2022.3178945).
- MAKIMOV, Kirill. **A qualitative case study on using TypeScript as a JavaScript alternative in frontend web development in the industry**. 2022. Dissertação (Mestrado) – University of Tartu, Tartu. Disponível em: <https://dspace.ut.ee/bitstreams/76796c1b-8490-4a20-8925-99b1533eeaa9/download>.

- MORORÓ, Jadson Faustino. **Um estudo comparativo entre JavaScript e TypeScript**. Sobral, 2024. Disponível em: https://repositorio.ufc.br/bitstream/riufc/78817/1/2024_tcc_jfmororo.pdf.
- PEDRYCZ, Witold; GOMIDE, Fernando. **An introduction to fuzzy sets: analysis and design**. Cambridge: MIT Press, 1998. p. 465.
- RICHARDS, Gregor; NARDELLI, Francesco Zappa; VITEK, Jan. To type or not to type: Quantifying detectable bugs in JavaScript. *In: PROCEEDINGS of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 2017. p. 758–769. Disponível em: <https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/gao2017javascript.pdf>.
- SHRESTHA, Nischal *et al.* Here we go again: Why is it difficult for developers to learn another programming language? **Communications of the ACM**, 2022. Disponível em: <https://cacm.acm.org/research/here-we-go-again/>.
- STACK OVERFLOW. **2023 Developer Survey**. 2023. Disponível em: <https://survey.stackoverflow.co/2023/>. Acesso em: 27 jul. 2025.
- TANG, Tianyi; ALIMADADI, Saba; SUMNER, Nick. From logic to toolchains: An empirical study of bugs in the TypeScript ecosystem. *In: PROCEEDINGS of the 23rd International Conference on Mining Software Repositories*. 2026. p. 539–549. DOI: [10.1145/3793302.3793379](https://doi.org/10.1145/3793302.3793379). Disponível em: <https://arxiv.org/abs/2601.21186>.
- WANG, Ziyuan; FANG, Yun; WANG, Nannan. An empirical study on bugs in TypeScript programming language. **Journal of Systems and Software**, v. 226, p. 112445, 2025. DOI: [10.1016/j.jss.2025.112445](https://doi.org/10.1016/j.jss.2025.112445).
- ZANDONOTTO, Thomaz *et al.* How TypeScript compiler options influence code quality: A correlation study with Git repositories. **Journal of Software Engineering Research and Development**, v. 14, n. 1, p. 228–246, 2026. DOI: [10.5753/jserd.2026.5985](https://doi.org/10.5753/jserd.2026.5985).
- ZIMMERMANN, H. J. **Fuzzy set theory**. New York: John Wiley & Sons, 2010. v. 2.